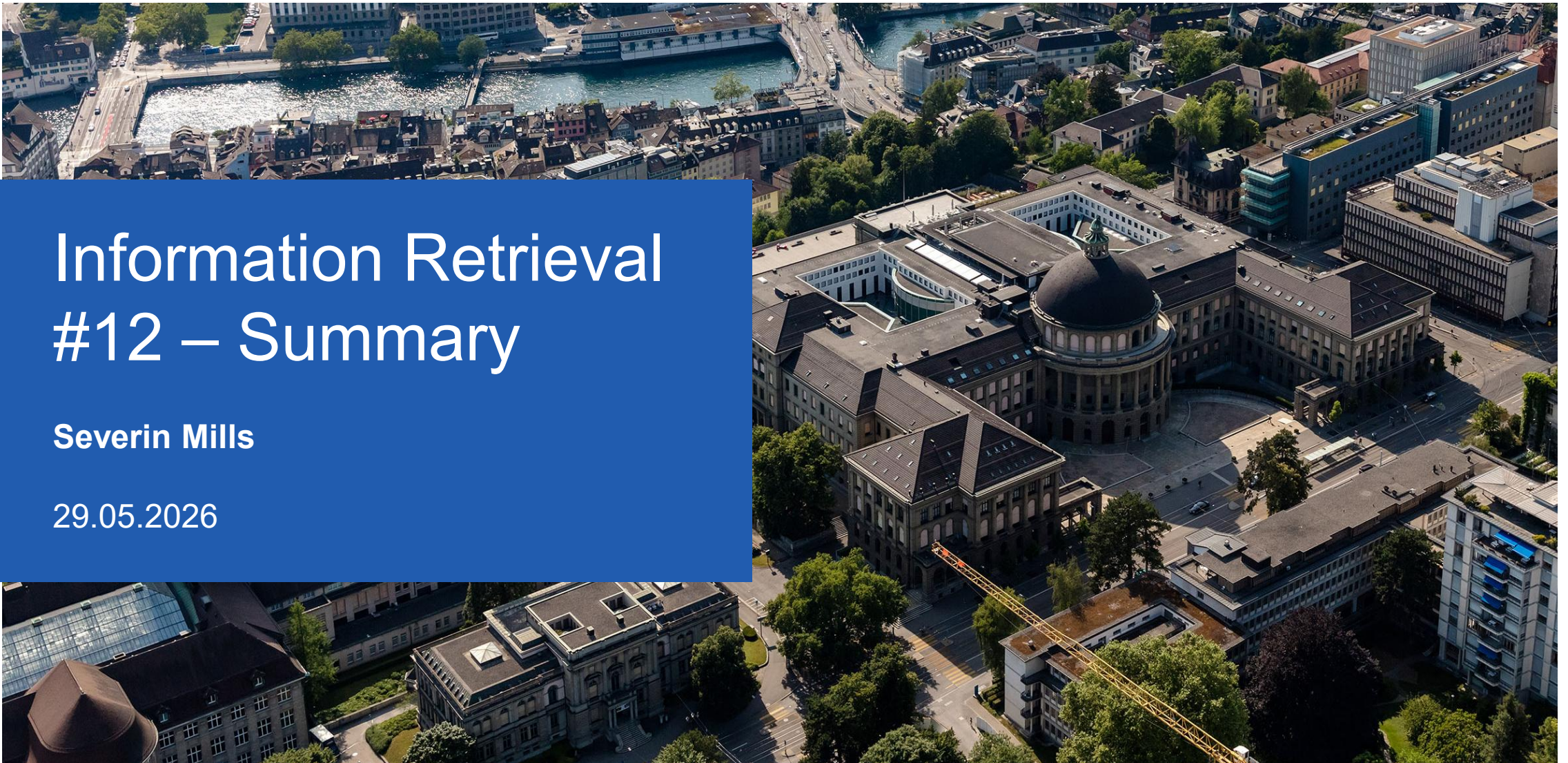


Information Retrieval #12 – Summary

Severin Mills

29.05.2026



Agenda

1. Semester Recap + Typical Exam Questions
2. Q&A

Incidence Matrix

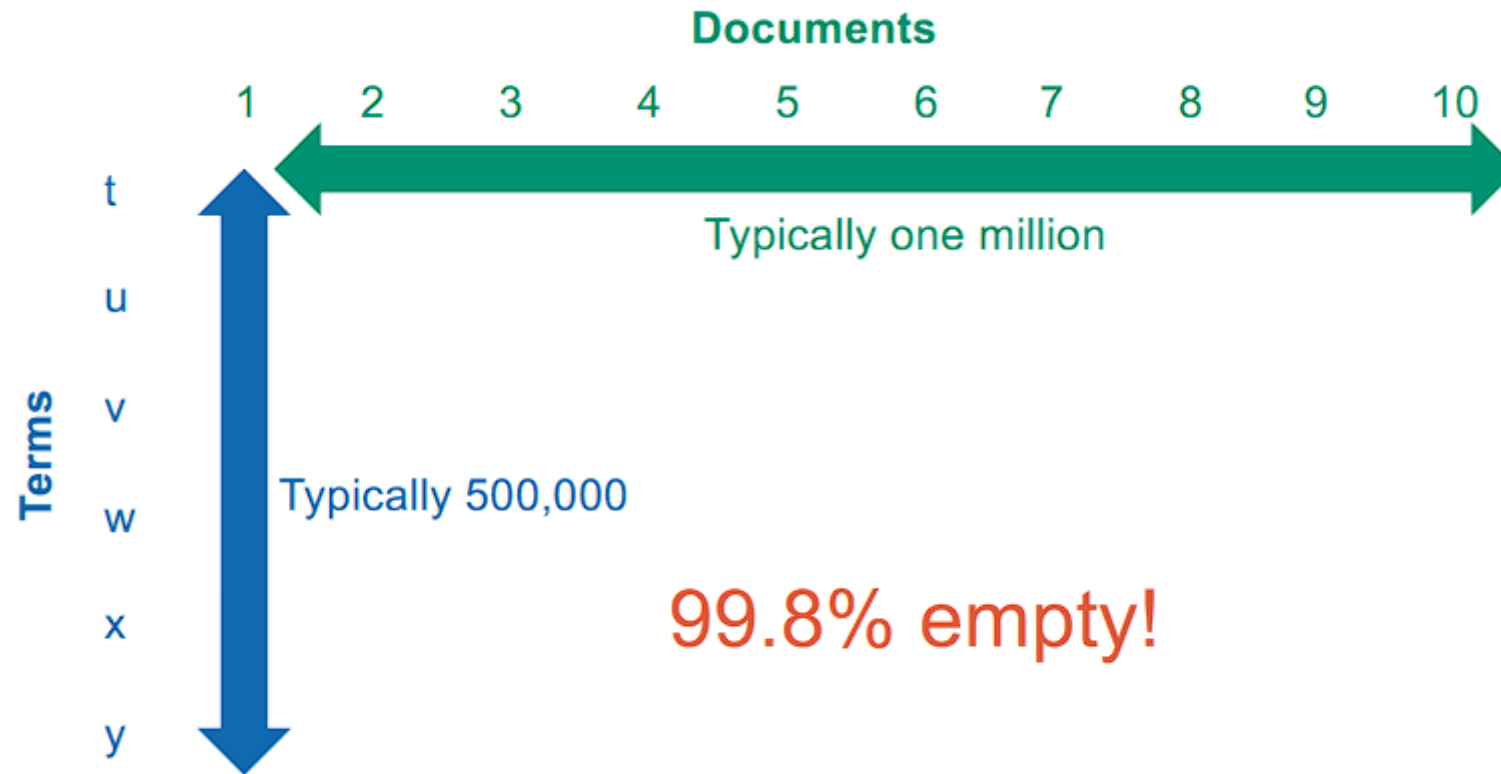
$$\text{contains}_{i,j} = \begin{cases} 1 & \text{if document } j \text{ contains term } i \\ 0 & \text{if document } j \text{ does not contain term } i \end{cases}$$

Query: u

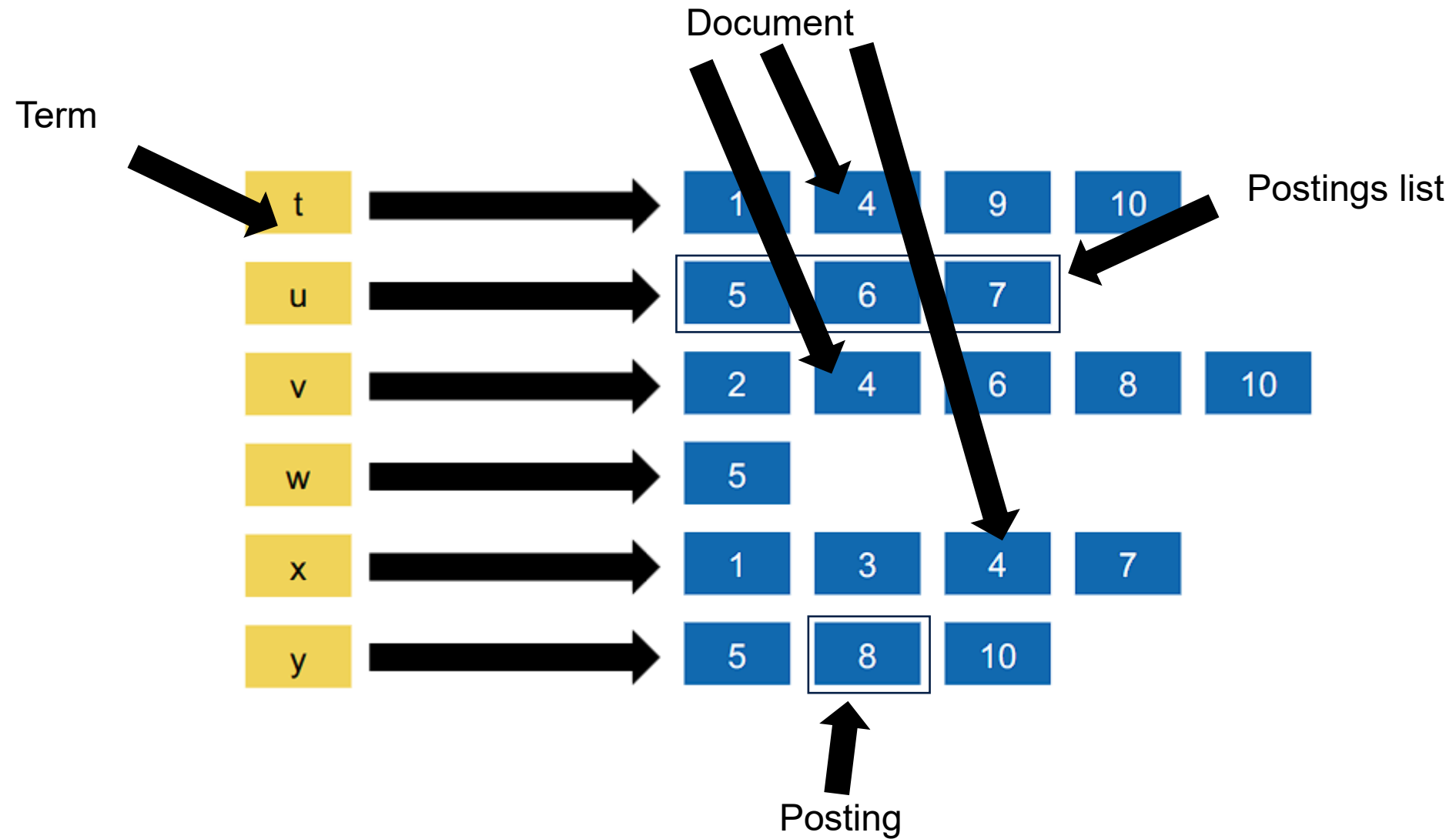
		Documents									
		1	2	3	4	5	6	7	8	9	10
Terms	t	1	0	1	1	1	0	0	0	1	1
	u	0	0	1	0	1	1	1	1	0	0
	v	0	1	1	1	0	1	0	1	0	1
	w	0	0	0	1	1	0	0	0	0	0
	x	1	0	1	1	1	0	1	0	0	1
	y	0	0	0	0	1	0	0	1	0	1

↑
 $\text{contains}_{y,5} = 1$

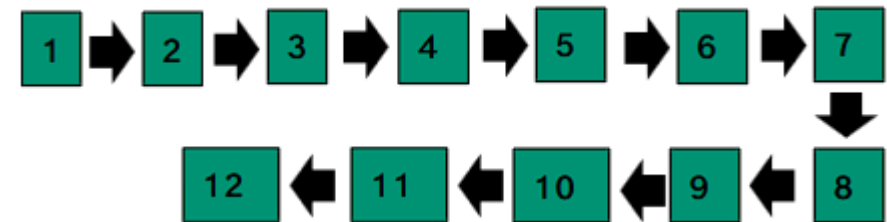
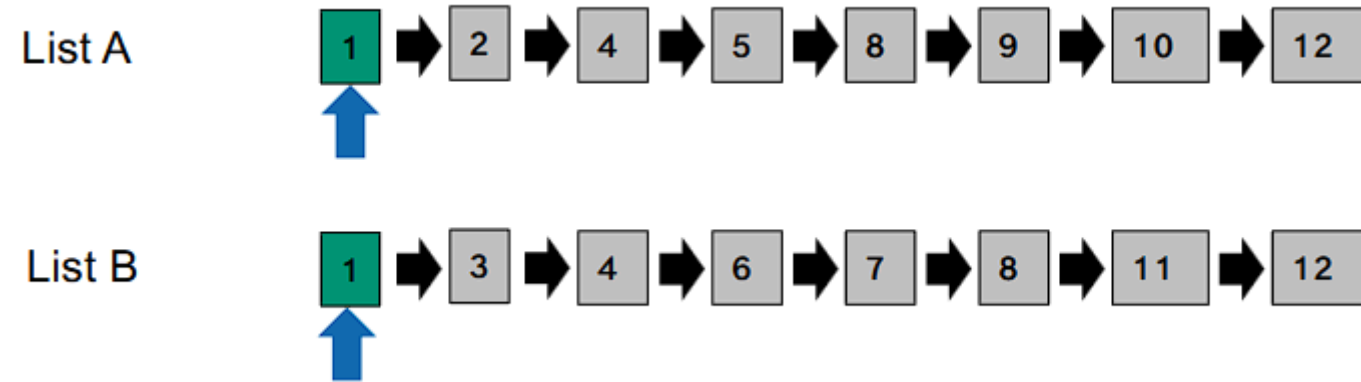
Incidence Matrix



Inverted Index



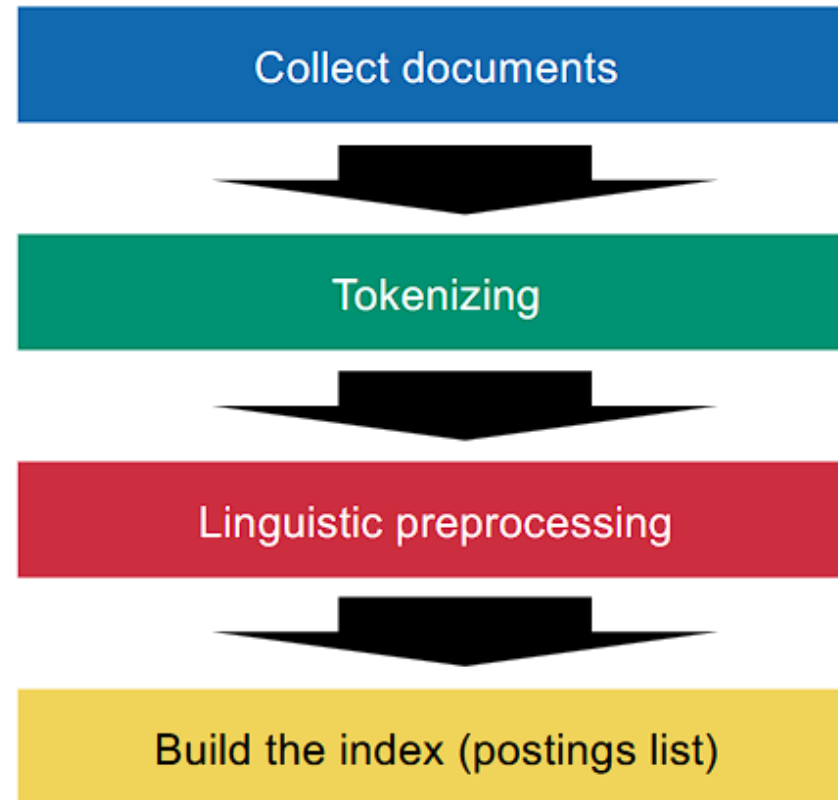
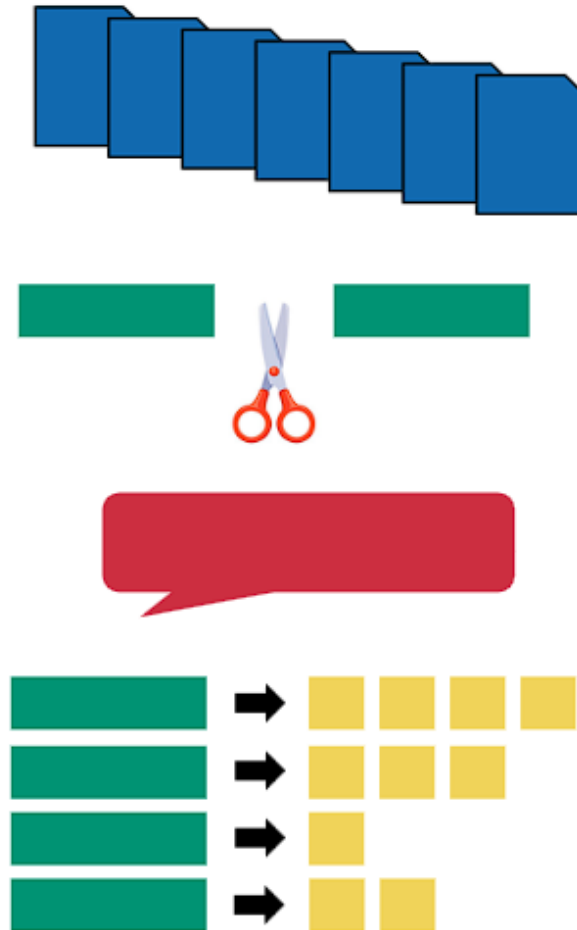
Intersection + Union Algorithm



Boolean Retrieval – Exam Questions

- Querying on Incidence Matrices
- Building the Inverted Index
- Querying on the Inverted Index
- Complexities of Query Types
- Intersection and Union Algorithm (steps, length)
 - Skip pointers

Index Construction is challenging...



Index Construction is challenging...

- Collecting documents
 - What language? What encoding? What context?
- Tokenization
 - Punctuation, Stop words, Lots of edge cases
 - Know this list:

Raw or processed	Tied to document	Full name	Simplified/casual
raw	tied with position	positional token	token (implicitly positional)
raw	tied without position	non-positional token	
raw	not tied	word, non-normalized type	type (implicitly non-normalized in the book) token (compiler community)
processed	tied with position	positional posting	
processed	tied without position	non-positional posting	posting (implicitly non-positional)
processed	not tied	normalized type, term (if in index)	

Index Construction is challenging...

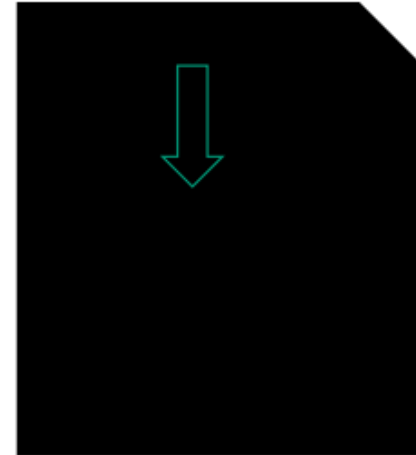
- Linguistic preprocessing
 - Normalization
 - Expansion
 - Stemming vs. Lemmatization

Phrase search

- Positional index vs. Bi-word index
 - Positional: Can reconstruct whole document
 - Bi-word: False Positives



Biword indices



Positional indices

Term Vocabulary – Exam Questions

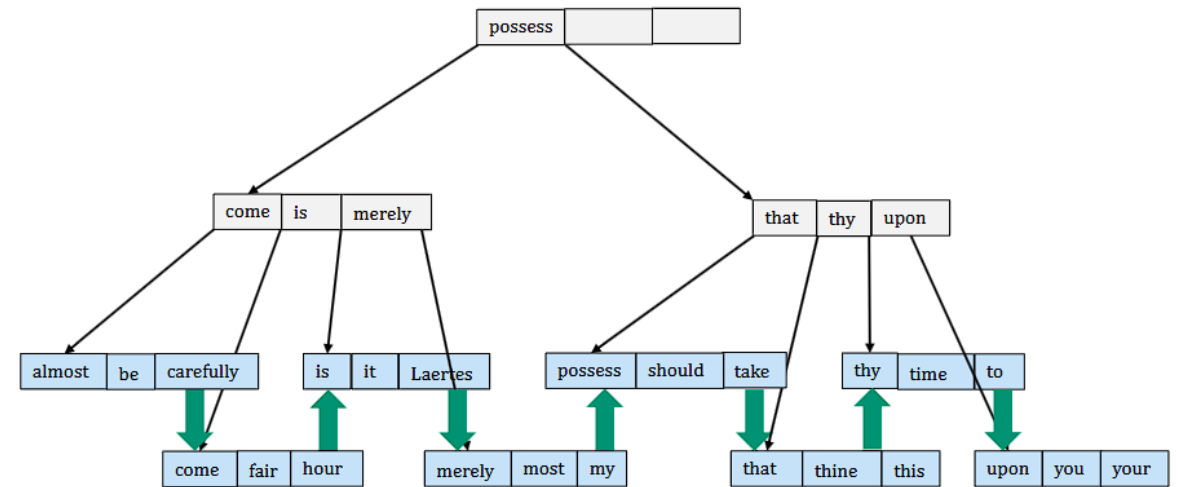
- Applying Stemming and Lemmatization
- Building a bi-word index
- Querying on bi-word and positional indices
- Reconstructing documents from positional indices

B+-Tree

A n - m B+-Tree has between n and m children and between $n - 1$ and $m - 1$ keys. Root may have less.

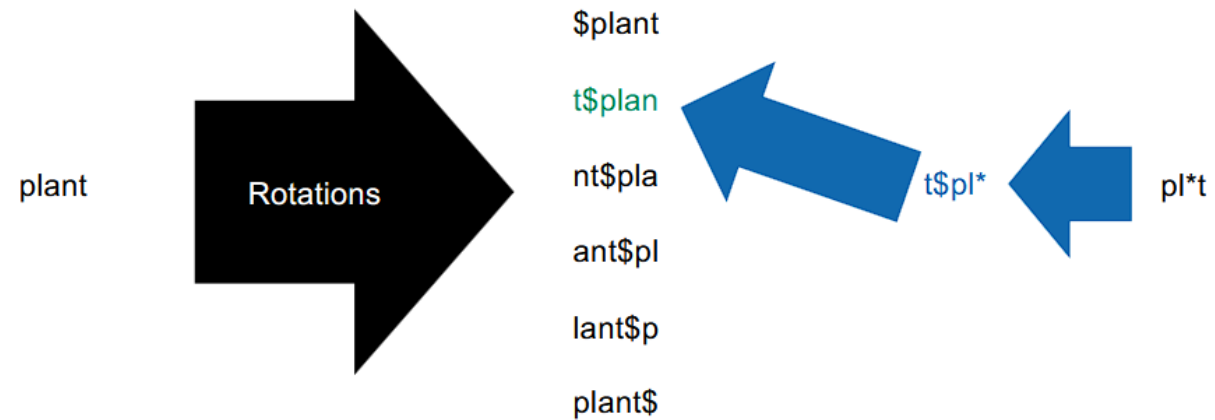
All leaves have the same depth.

Usually, they have extra pointers in the posting lists.



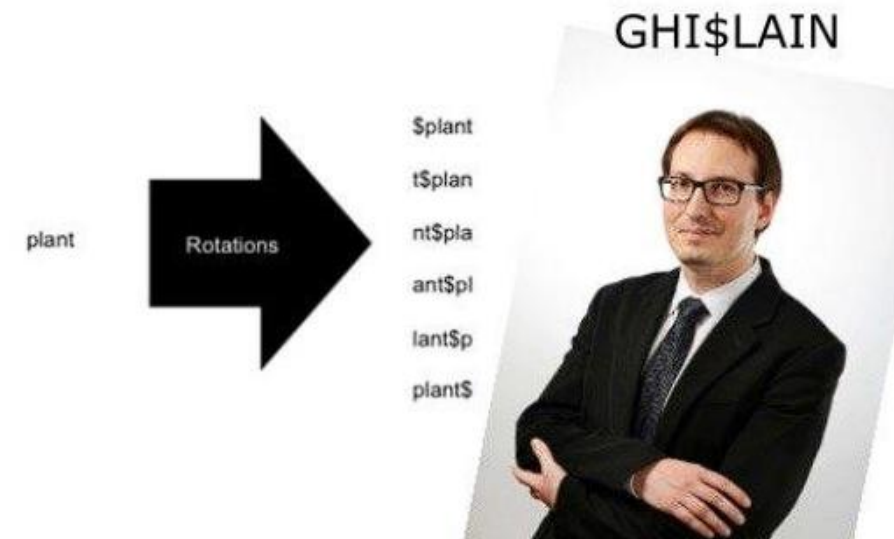
Permuterm Index

Use a B+-Tree to store all rotations

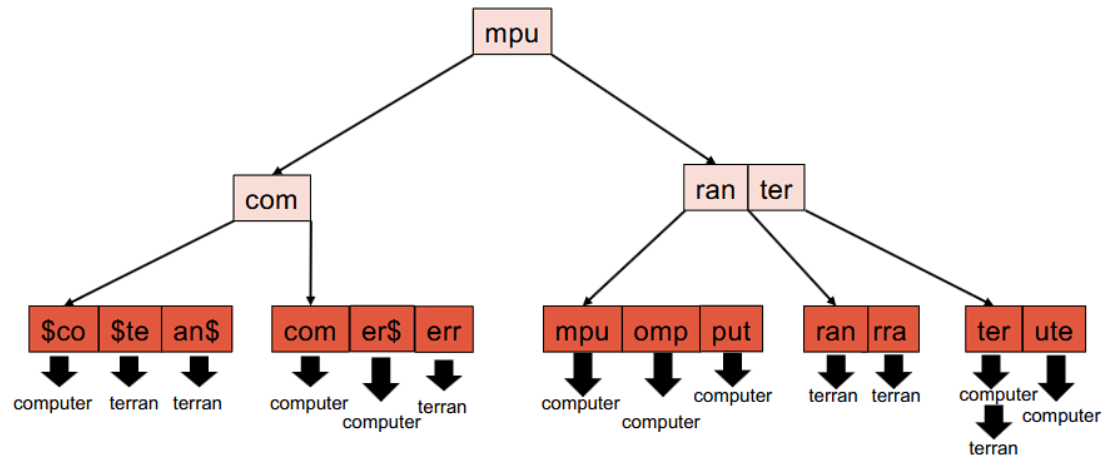


CHOOSE YOUR RAPPER NAME:

Permuterm index



K-gram Index



computer

1-grams \$, c, o, m, p, u, t, e, r, \$ **Not very useful**

2-grams **\$c, co, om, mp, pu, ut, te, er, r\$**

3-grams **\$co, com, omp, mpu, put, ute, ter, er\$**

4-grams **\$com, comp, ompu, mput, pute, uter, ter\$**

5-grams \$comp, compu, omput, mpute, puter, uter\$

6-grams \$compu, comput, ompute, mputer, puter\$

7-grams \$comput, compute, omputer, mputer\$

...

Usable
zone

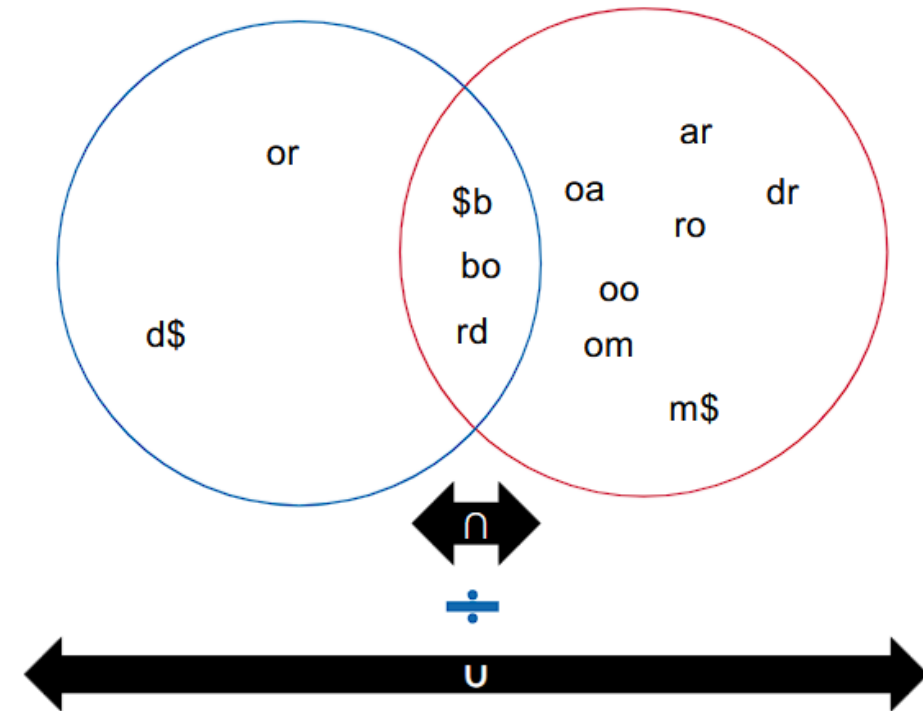
**Not space
efficient**

Spell Correction

	#	a	t	e
#	0	1	2	3
c	1	1	2	3
a	2			
t	3			

Annotations for Edit Distance calculation:
 - From (c, #) to (a, #): 0 (do nothing)
 - From (c, a) to (a, a): +1 (add a)
 - From (c, t) to (a, t): min(3, 1, 2)
 - From (a, t) to (a, t): +1 (remove a)

Edit Distance



Jaccard Coefficient

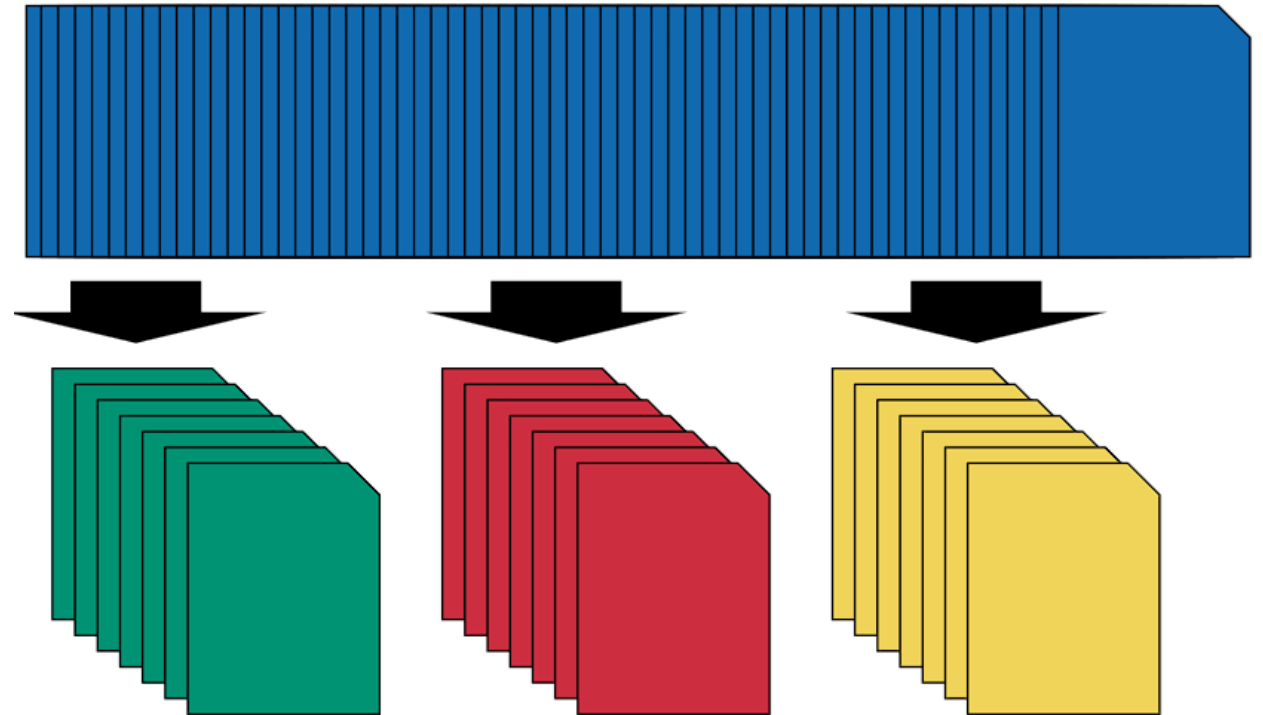
Tolerant Retrieval – Exam Questions

- Calculations using Jaccard Coefficient and Edit Distance
- Working with k-grams, especially recognizing false positives
- Working with permuterm indices

Blocked Sort-Based Indexing (BSBI)

1. Shard the collection of documents
2. Process each block one-by-one in memory
 - Parse termID-docID pairs
 - Sort pairs according to termID
 - Write back intermediate results

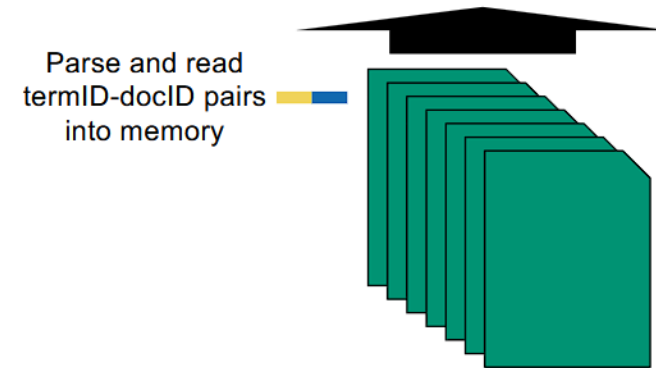
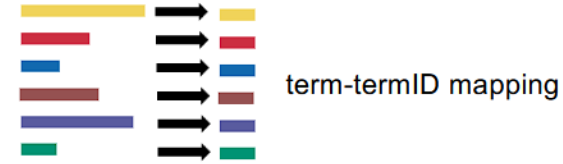
Complexity: $\mathcal{O}(T \log T)$



Blocked Sort-Based Indexing (BSBI)

1. Shard the collection of documents
2. Process each block one-by-one in memory
 - Parse termID-docID pairs
 - Sort pairs according to termID
 - Write back intermediate results

Complexity: $\mathcal{O}(T \log T)$



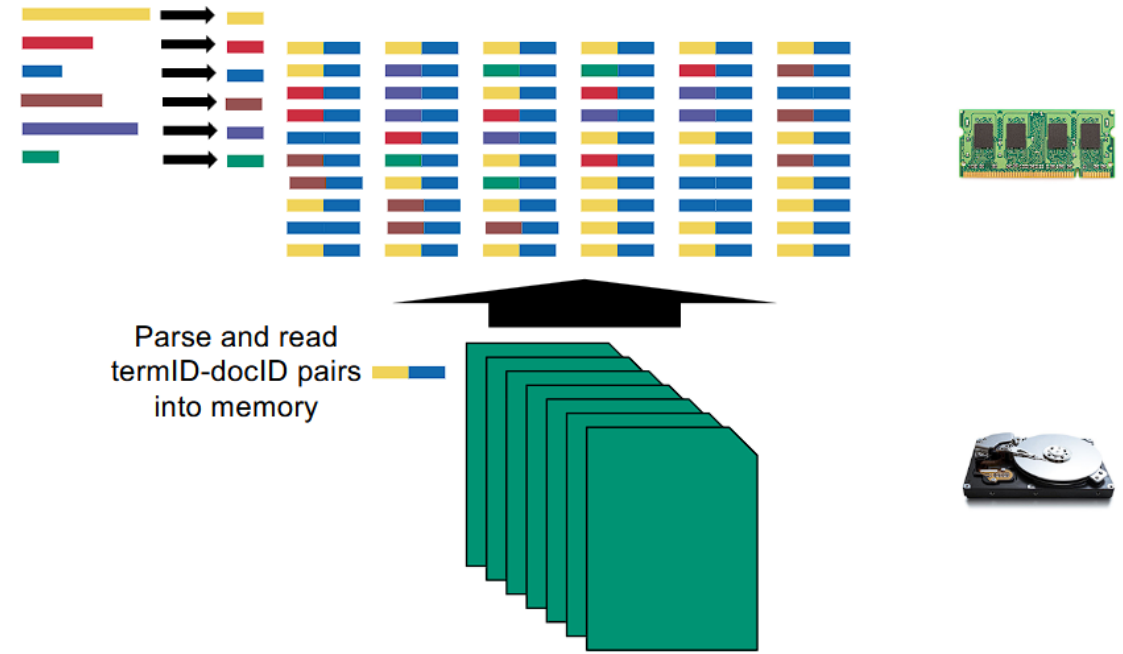
Blocked Sort-Based Indexing (BSBI)

1. Shard the collection of documents
2. Process each block one-by-one in memory
 - Parse termID-docID pairs
 - Sort pairs according to termID
 - Write back intermediate results

Complexity: $\mathcal{O}(T \log T)$

Generating term-termID mappings:

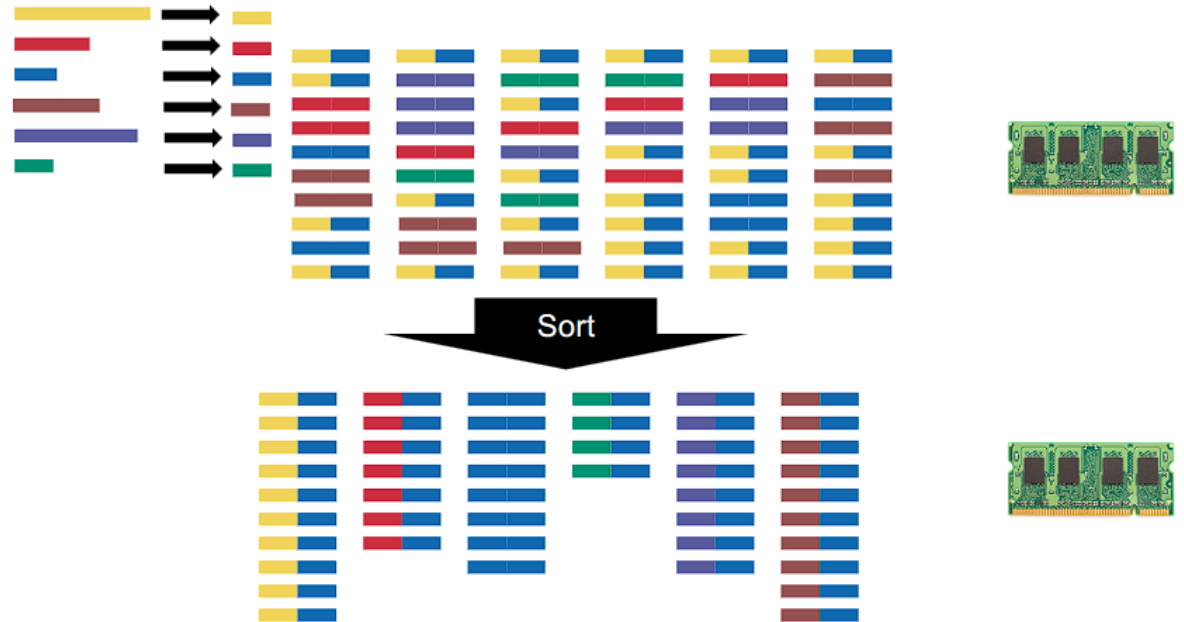
- On-the-fly vs. First pass



Blocked Sort-Based Indexing (BSBI)

1. Shard the collection of documents
2. Process each block one-by-one in memory
 - Parse termID-docID pairs
 - Sort pairs according to termID
 - Write back intermediate results

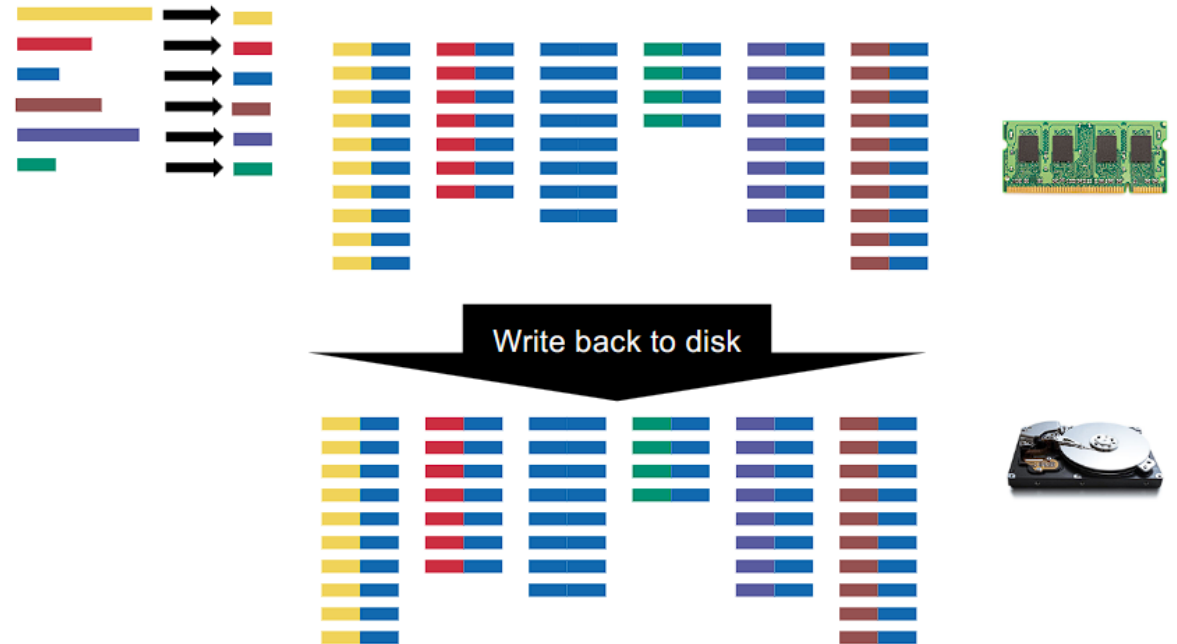
Complexity: $\mathcal{O}(T \log T)$



Blocked Sort-Based Indexing (BSBI)

1. Shard the collection of documents
2. Process each block one-by-one in memory
 - Parse termID-docID pairs
 - Sort pairs according to termID
 - Write back intermediate results

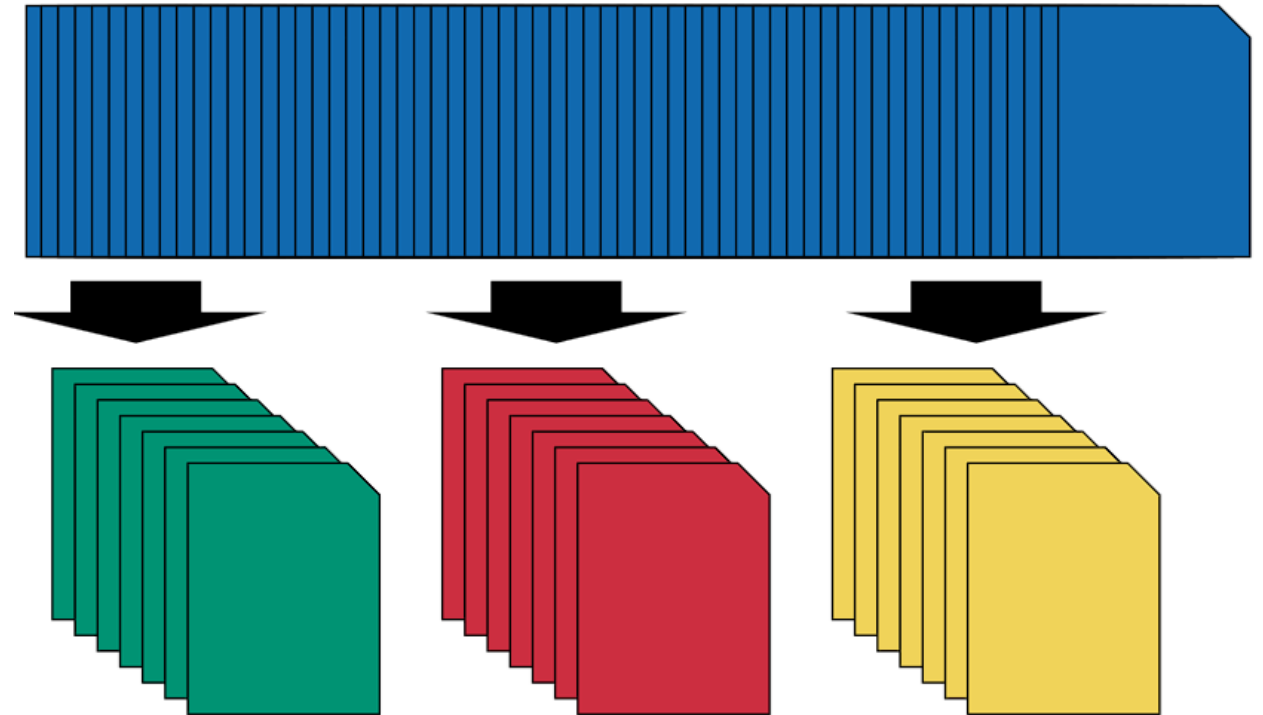
Complexity: $\mathcal{O}(T \log T)$



Single-Pass In-Memory Indexing (SPIMI)

1. Shard the collection of documents
2. Process each block one-by-one in memory
 - Create intermediary index
 - Sort index by term
 - Write back intermediate index to disk

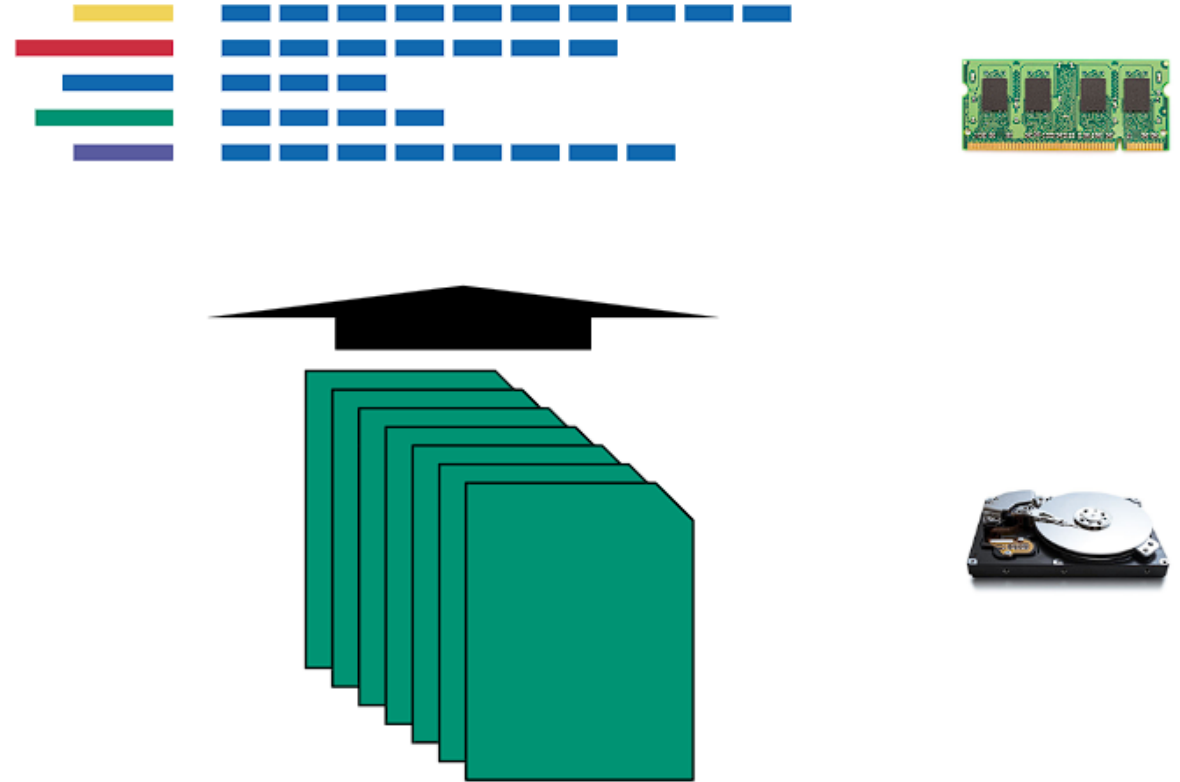
Complexity: $\mathcal{O}(T \log M)$



Single-Pass In-Memory Indexing (SPIMI)

1. Shard the collection of documents
2. Process each block one-by-one in memory
 - Create intermediary index
 - Sort index by term
 - Write back intermediate index to disk

Complexity: $\mathcal{O}(T \log M)$



Single-Pass In-Memory Indexing (SPIMI)

1. Shard the collection of documents
2. Process each block one-by-one in memory
 - Create intermediary index
 - Sort index by term
 - Write back intermediate index to disk

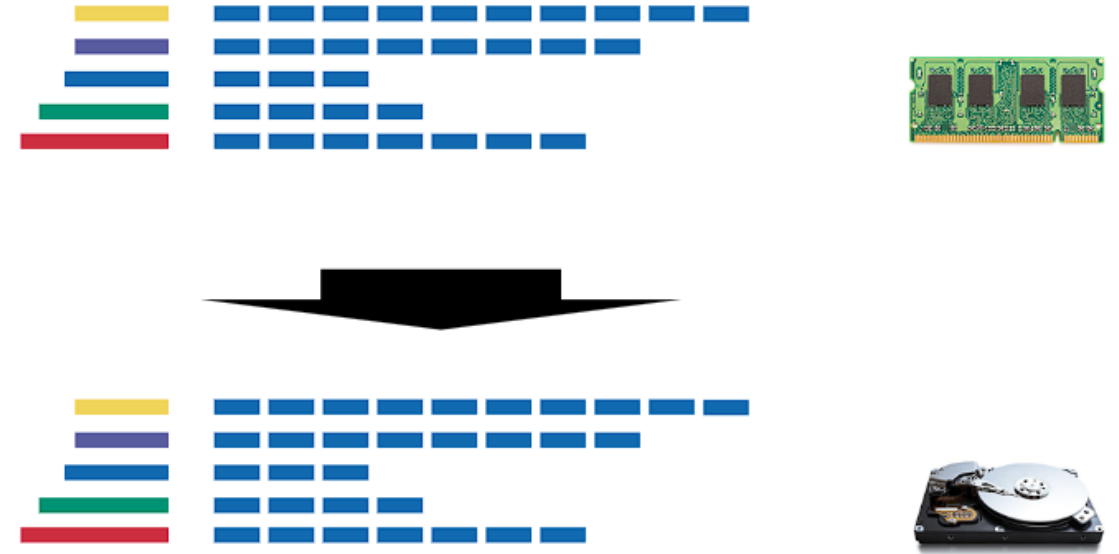


Complexity: $\mathcal{O}(T \log M)$

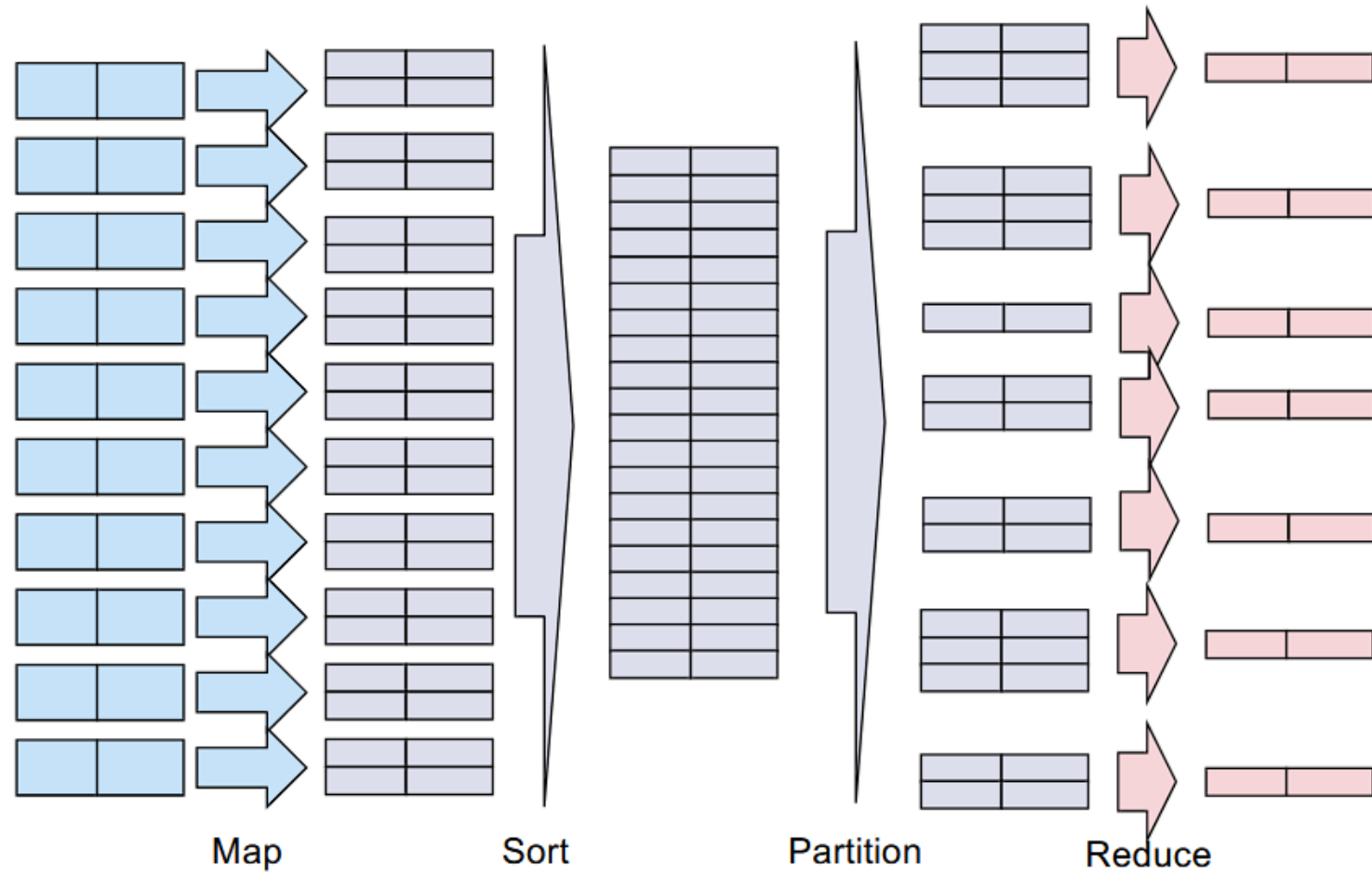
Single-Pass In-Memory Indexing (SPIMI)

1. Shard the collection of documents
2. Process each block one-by-one in memory
 - Create intermediary index
 - Sort index by term
 - Write back intermediate index to disk

Complexity: $\mathcal{O}(T \log M)$



MapReduce



Index Construction – Exam Questions

- Complexities and Algorithms of BSBI, SPIMI
- Know how MapReduce works
- Perform logarithmic merging

Index Compression

Heap's Law, Zipf's Law

$$M = k\sqrt{T}$$

$$\text{Frequency} = \frac{k}{\text{Rank}}$$

Compression methods:

UTF-8

Variable-length encoding

Fixed-length encoding

Unary Encoding

Gamma Encoding

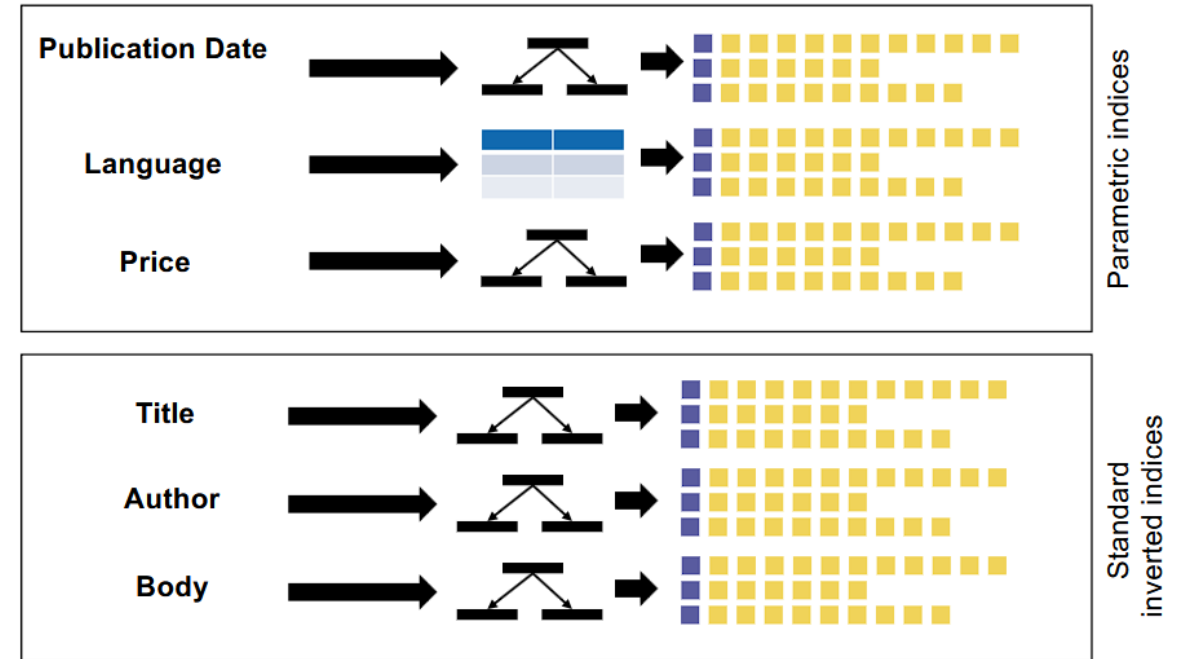
Encoding Gaps

Index Compression – Exam Questions

- Applying Heap's and Zipf's law in calculations
- Applying the various encoding methods (UTF-8, Variable, Fixed-length, Unary, Gamma)
- Gap encoding
- Check slides of that week for tricks in identifying encoding methods / lengths of encoding methods

Zone search

- Parametric indices: Each parameter gets an index
- Shared index: All parameters in one index, but each posting is flagged by parameter
- Both: Add weights to zone, rank by weights



Weights

- Term Frequency: How many times does a term appear in a document
 - Depends on term and document
- Collection Frequency: How many times does a term appear in a collection of documents
 - Depends on term
- Document Frequency: In how many documents does a term appear?
 - Depends on term
- Inverse Document Frequency: $\log \frac{N}{df_t}$, where the number of documents in the collection is N.
 - Depends on term
- Most common metric for scoring: tf-idf

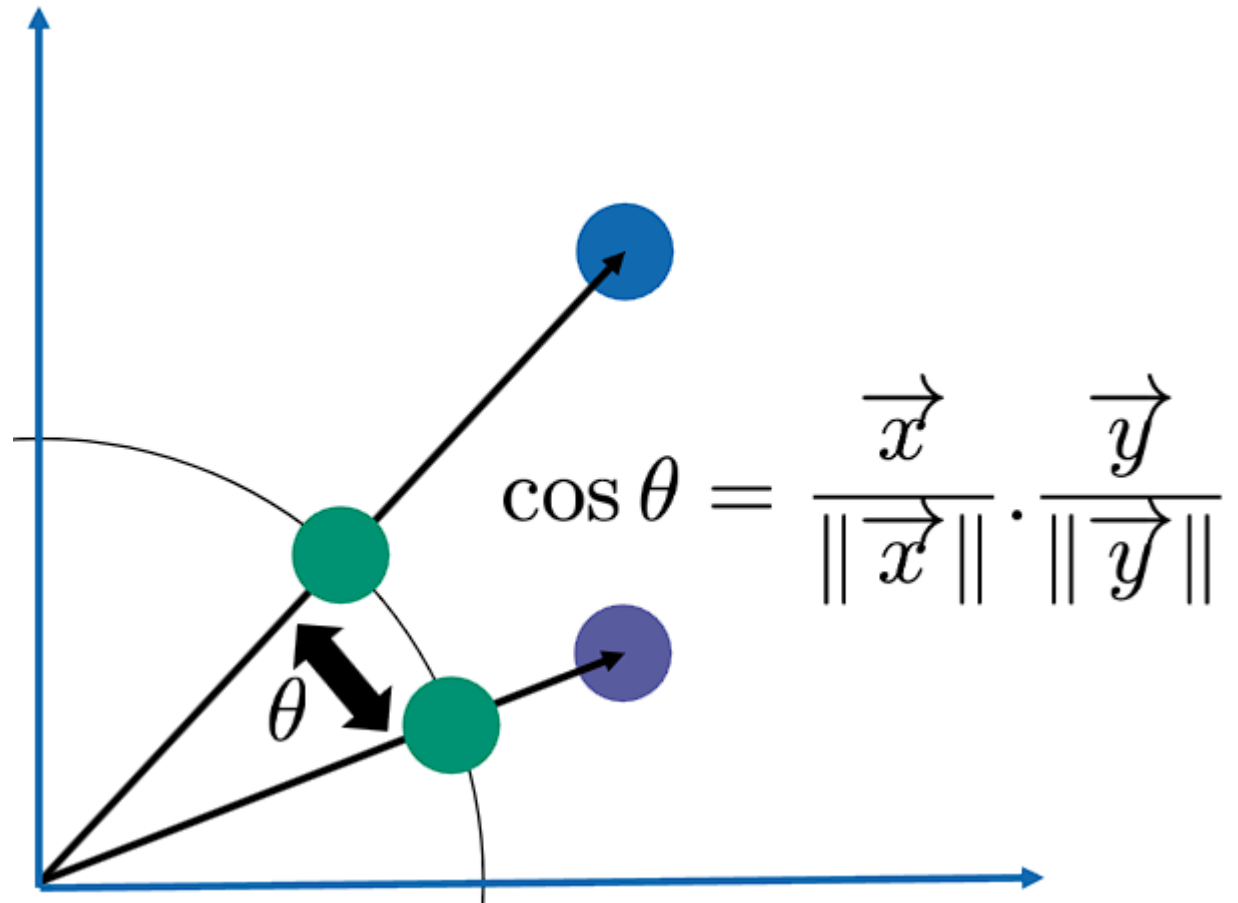
Ranked Retrieval – Exam Questions

- Querying on parametric indices and zone search
- Know how to use various weights (tf, idf, df, cf, tf-idf)

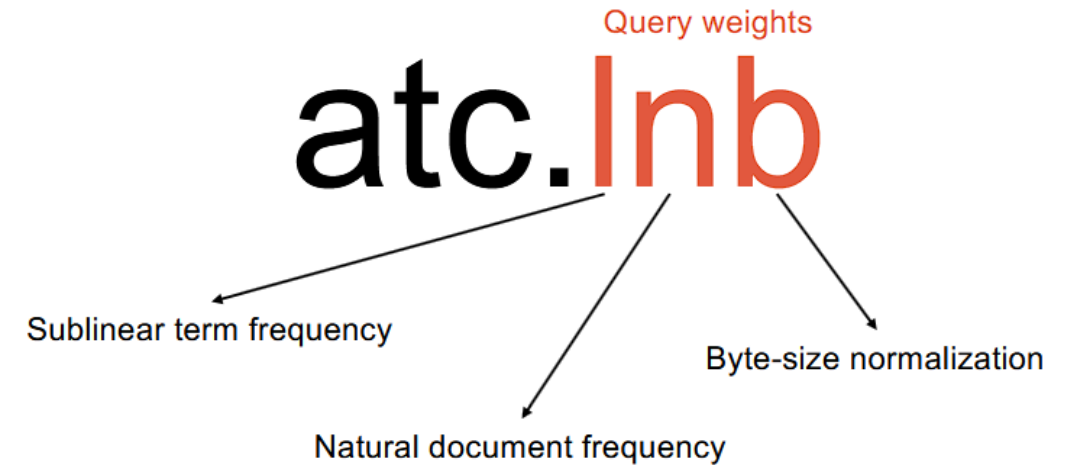
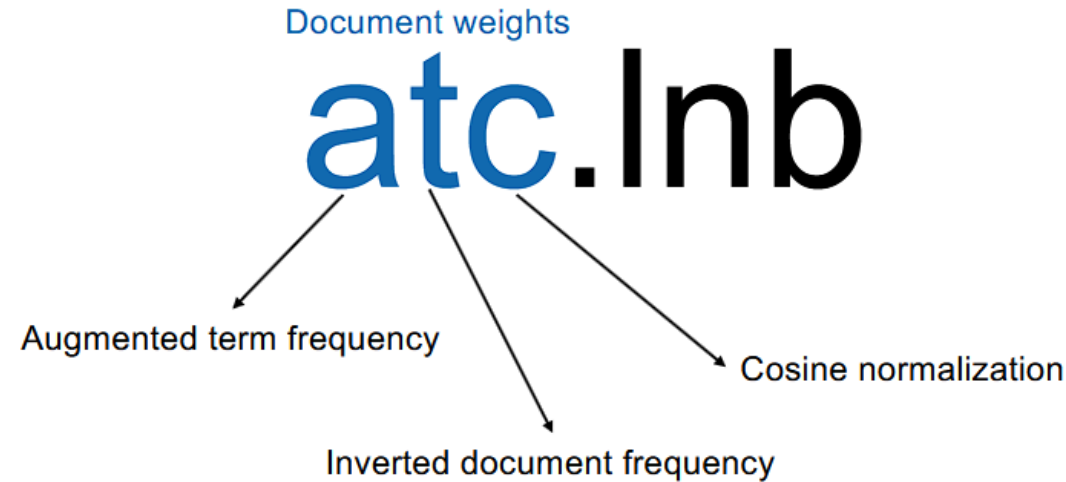
Vector Space Model

Encode documents and queries as vectors (using e.g. tf-idf as weights). Renormalized inner product gives similarity between vectors in the vector space.

$$\text{sim}(\vec{d}, \vec{q}) = \sum_t \frac{\text{tf-idf}_{td} \times \text{tf-idf}_{tq}}{\|\vec{q}\| \times \|\vec{d}\|}$$

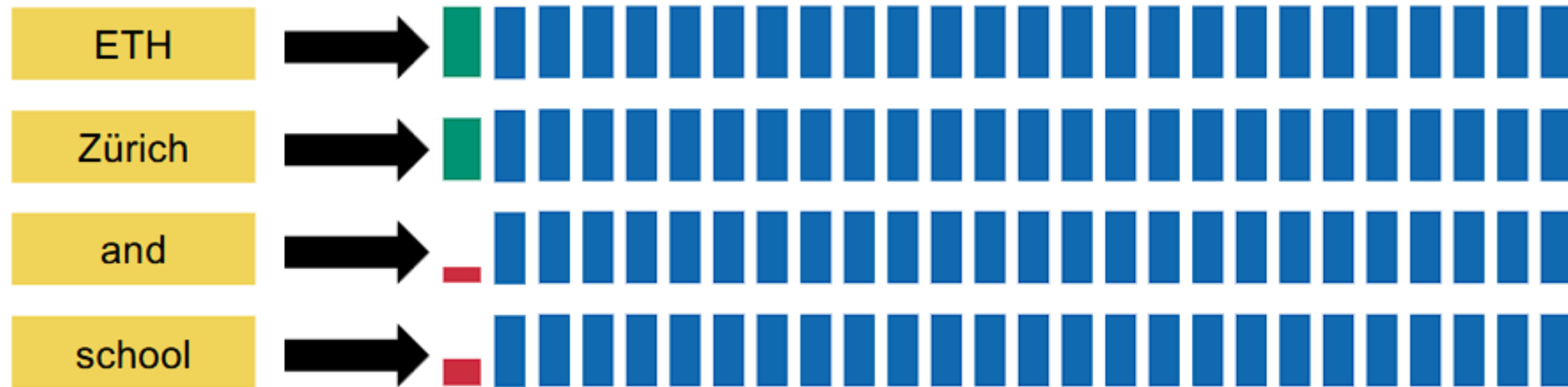


SMART Notation

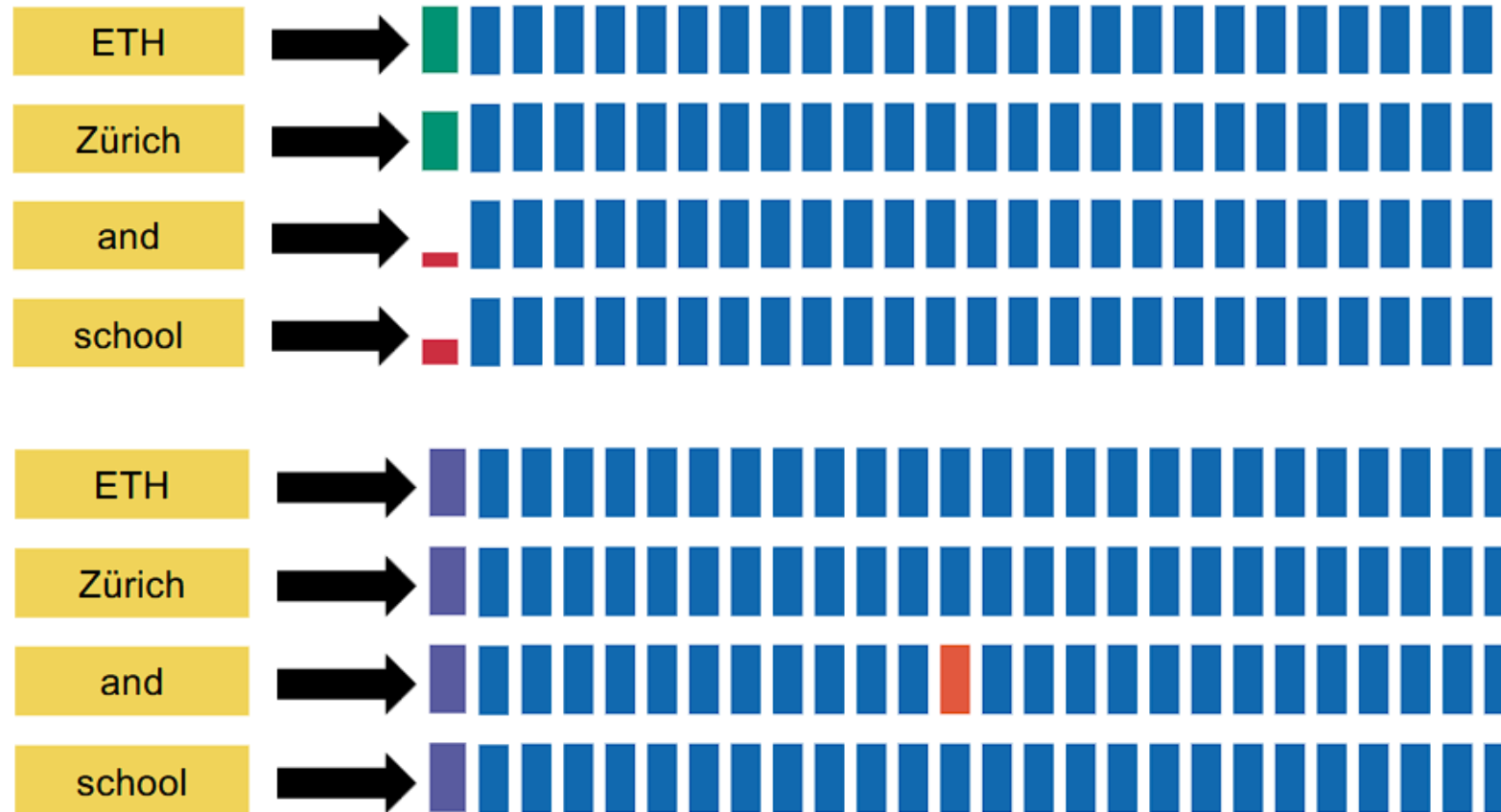


Term frequency		Document frequency		Normalization	
n (natural)	$tf_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1 + \log(tf_{t,d})$	t (idf)	$\log \frac{N}{df_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2 + w_2^2 + \dots + w_M^2}}$
a (augmented)	$0.5 + \frac{0.5 \times tf_{t,d}}{\max_t(tf_{t,d})}$	p (prob idf)	$\max\{0, \log \frac{N - df_t}{df_t}\}$	u (pivoted unique)	$1/u$ (Section 6.4.4)
b (boolean)	$\begin{cases} 1 & \text{if } tf_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$			b (byte size)	$1/CharLength^\alpha, \alpha < 1$
L (log ave)	$\frac{1 + \log(tf_{t,d})}{1 + \log(\text{ave}_{t \in d}(tf_{t,d}))}$				

Index Elimination



Index Elimination

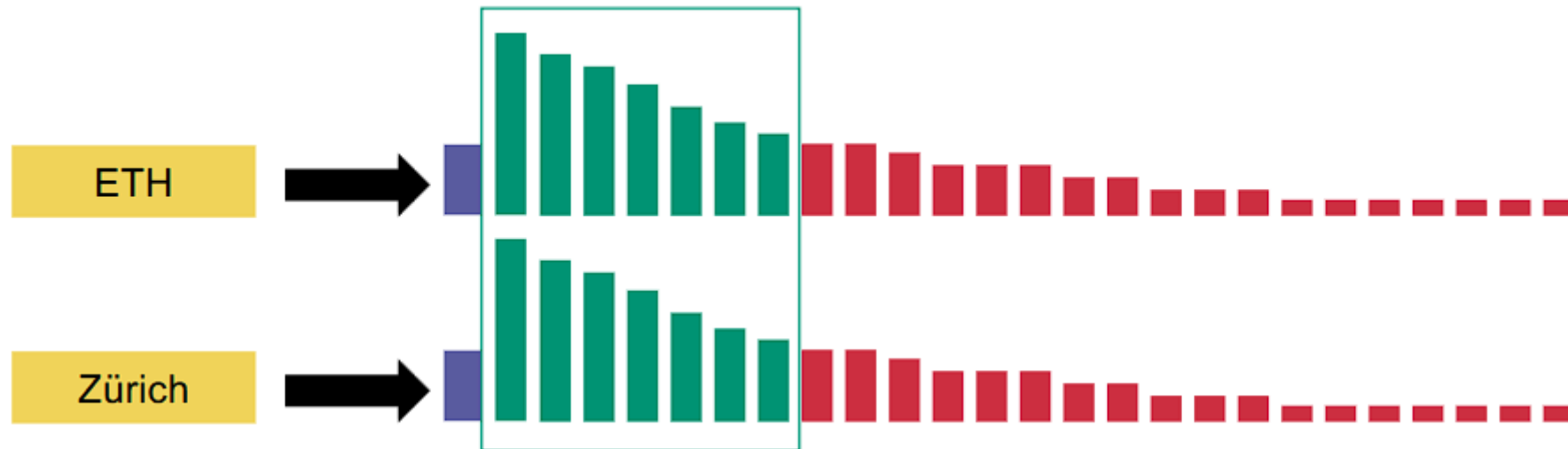


Drop documents containing only a few terms

(Easy to figure out using intersection algorithm)

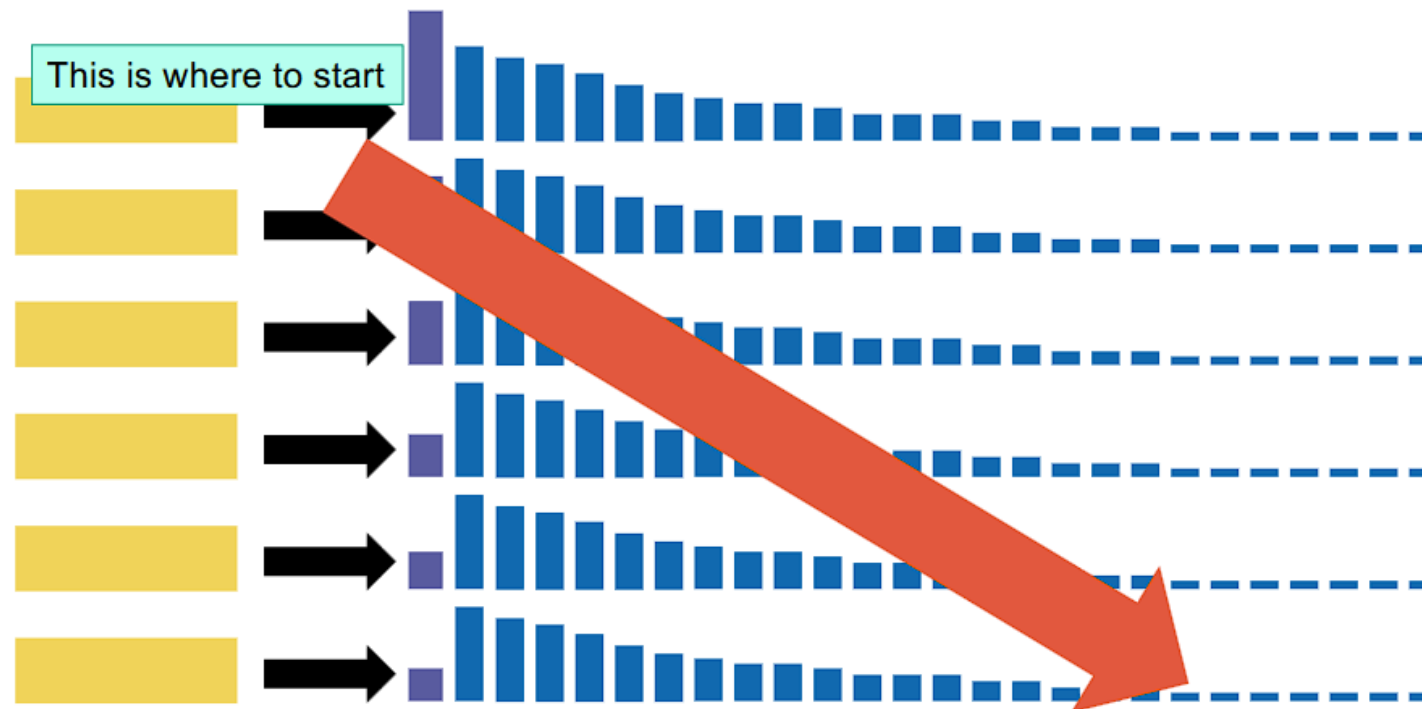
Champion Lists

1. Sort PLs by decreasing term frequency
2. Keep top r documents from each postings list
3. Union top r from each term



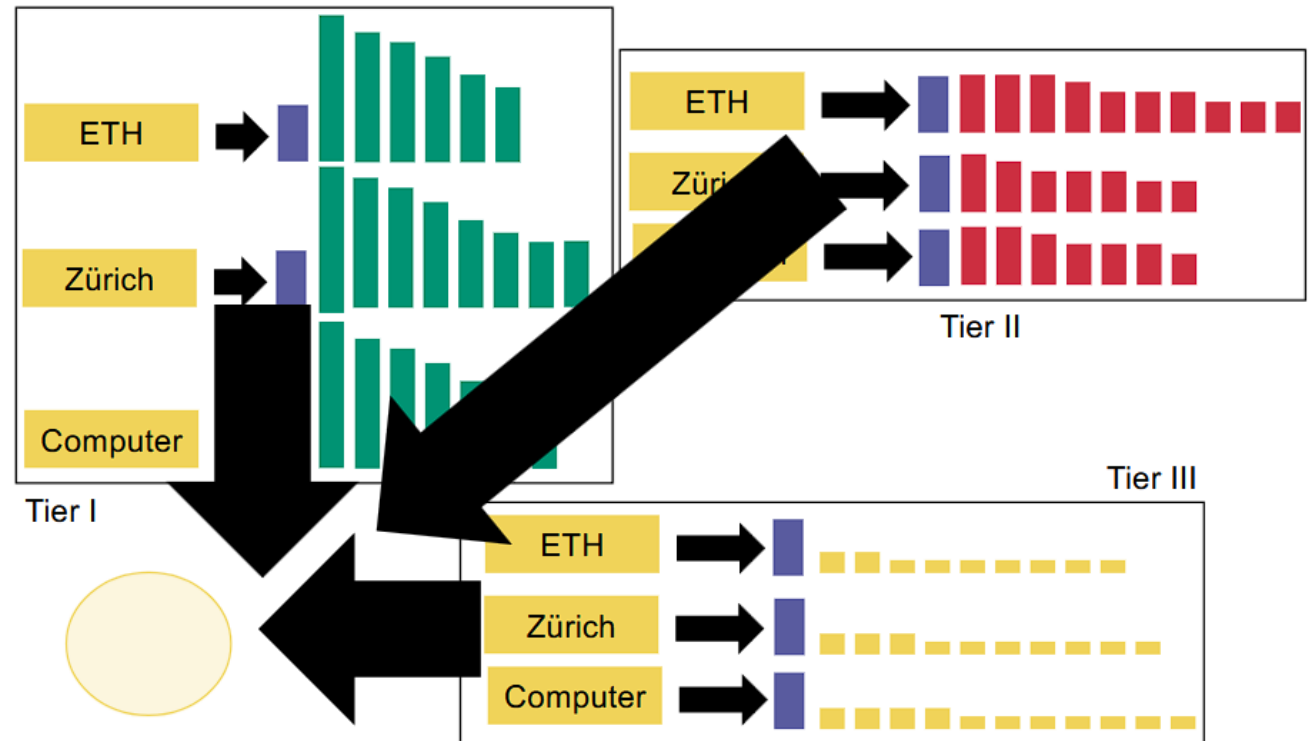
Impact ordering

1. Sort PLs by term frequency
2. Sort terms by inverted document frequency
3. Traverse term-at-a-time



Tiered indices

1. Build Champion Lists
2. Split up into tiers by term frequencies
3. Go through T1, if not enough documents, go through T2, etc.



Vector Space Model – Exam Questions

- How to apply normalization and cosine similarity
- Calculate similarity between query and document vectors
- SMART Notation (learn the basic ones)
- Various methods: Index Elimination, Champion Lists, Impact ordering, Tiered indices, Clustering

Evaluation Metrics

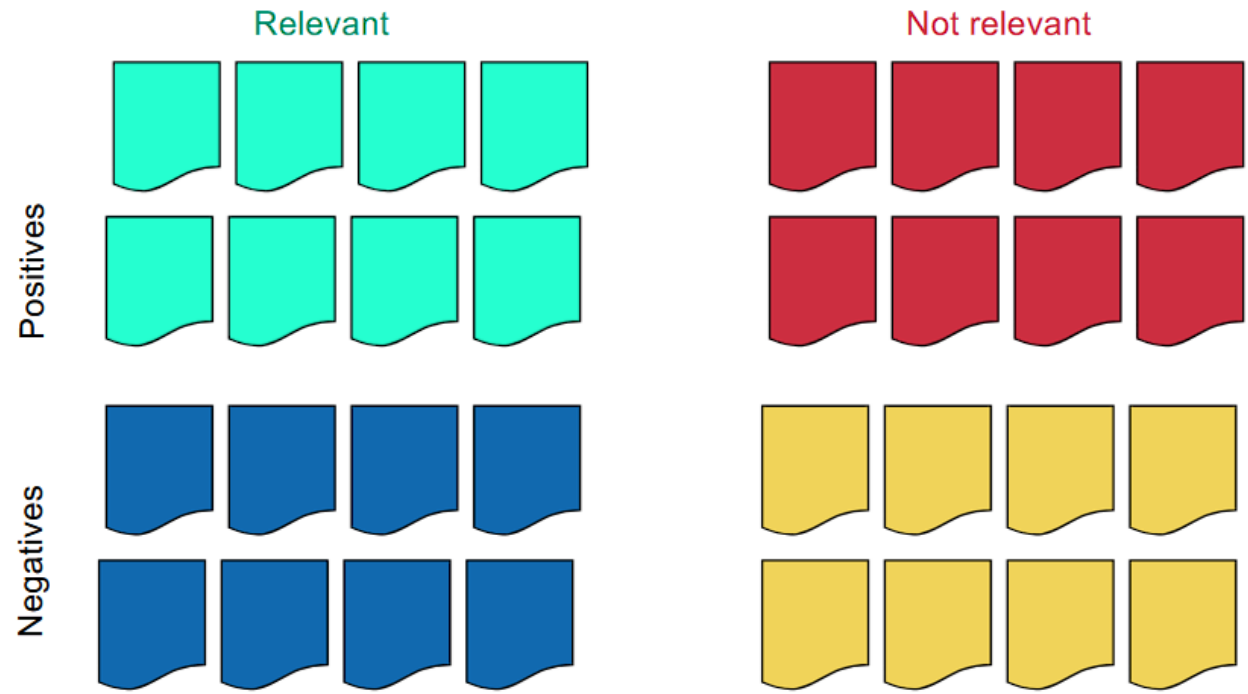
Recall: $\frac{\text{True positives}}{\text{Relevant}}$, Precision: $\frac{\text{True positives}}{\text{Returned}}$, Specificity: $\frac{\text{True negatives}}{\text{Irrelevant}}$, Accuracy: $\frac{\text{TP} + \text{TN}}{\text{All Results}}$

F-Measure: $F_{\alpha} = \frac{1}{\frac{\alpha}{P} + \frac{1-\alpha}{R}}$

R-Precision: The point in the Precision-Recall Graph where Precision=Recall.

Mean-Average Precision: Averaging the Precision over multiple queries.

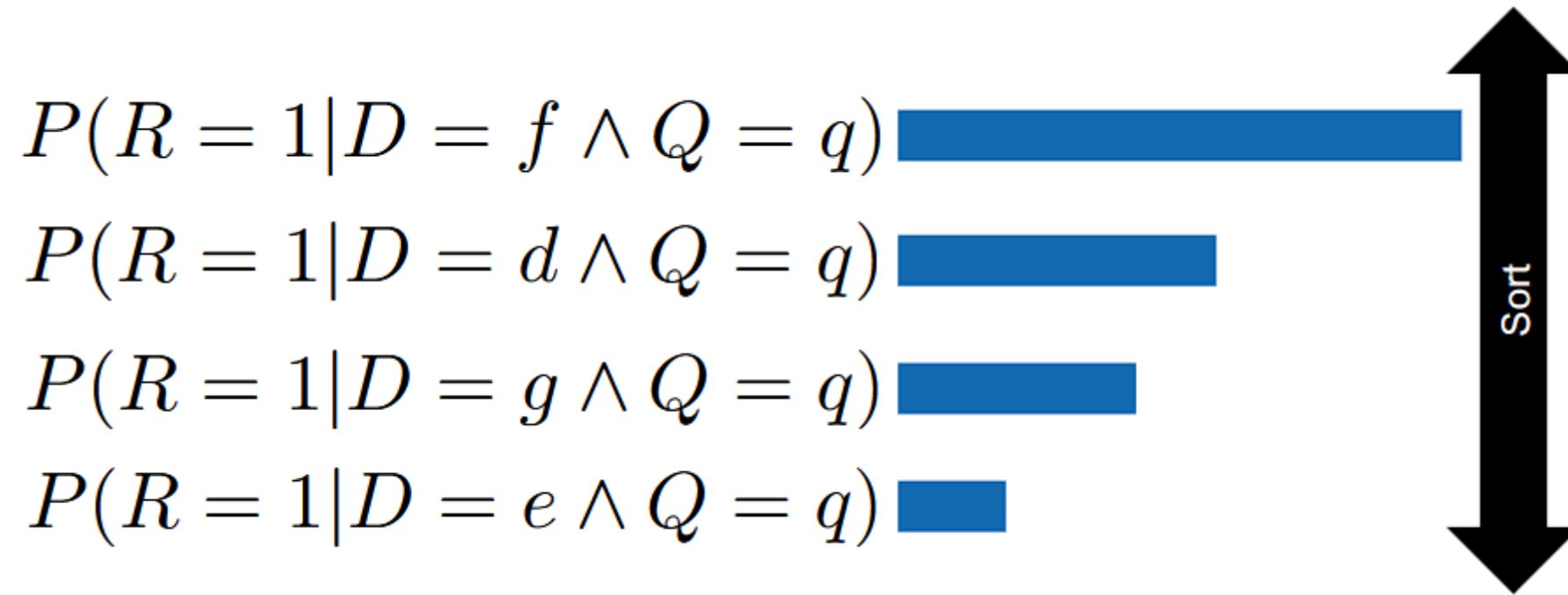
ROC Curve: Plotting Recall vs. Sensitivity



Evaluation – Exam Questions

- Know the different metrics (Recall, Precision, Specificity, Accuracy)
- Know the different error types (T1, T2, FP, FN)
- F-Measure
- Understand ROC Curve and R-Precision

Probability Ranking Principle



Retrieval Status Value

With a few simplifying assumptions, the weights in the RSV can be estimated back to the idf.

$$RSV_d = \sum_{k|d_k=1 \wedge q_k=1} \log \frac{p_k}{1 - p_k} - \log \frac{u_k}{1 - u_k}$$

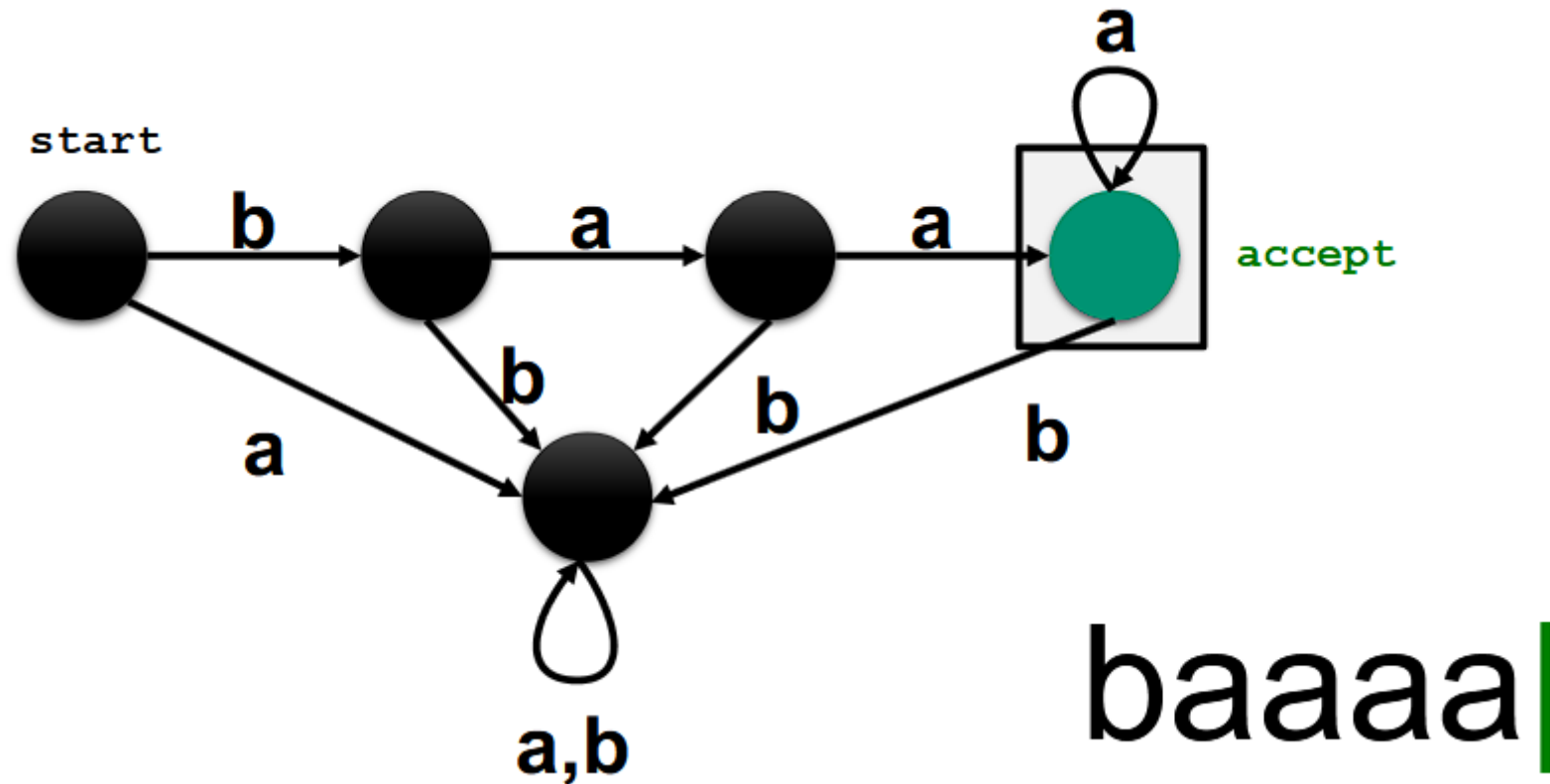
Odds of containing term k
in relevant documents

Odds of containing term k
in non-relevant documents

Probabilistic Retrieval – Exam Questions

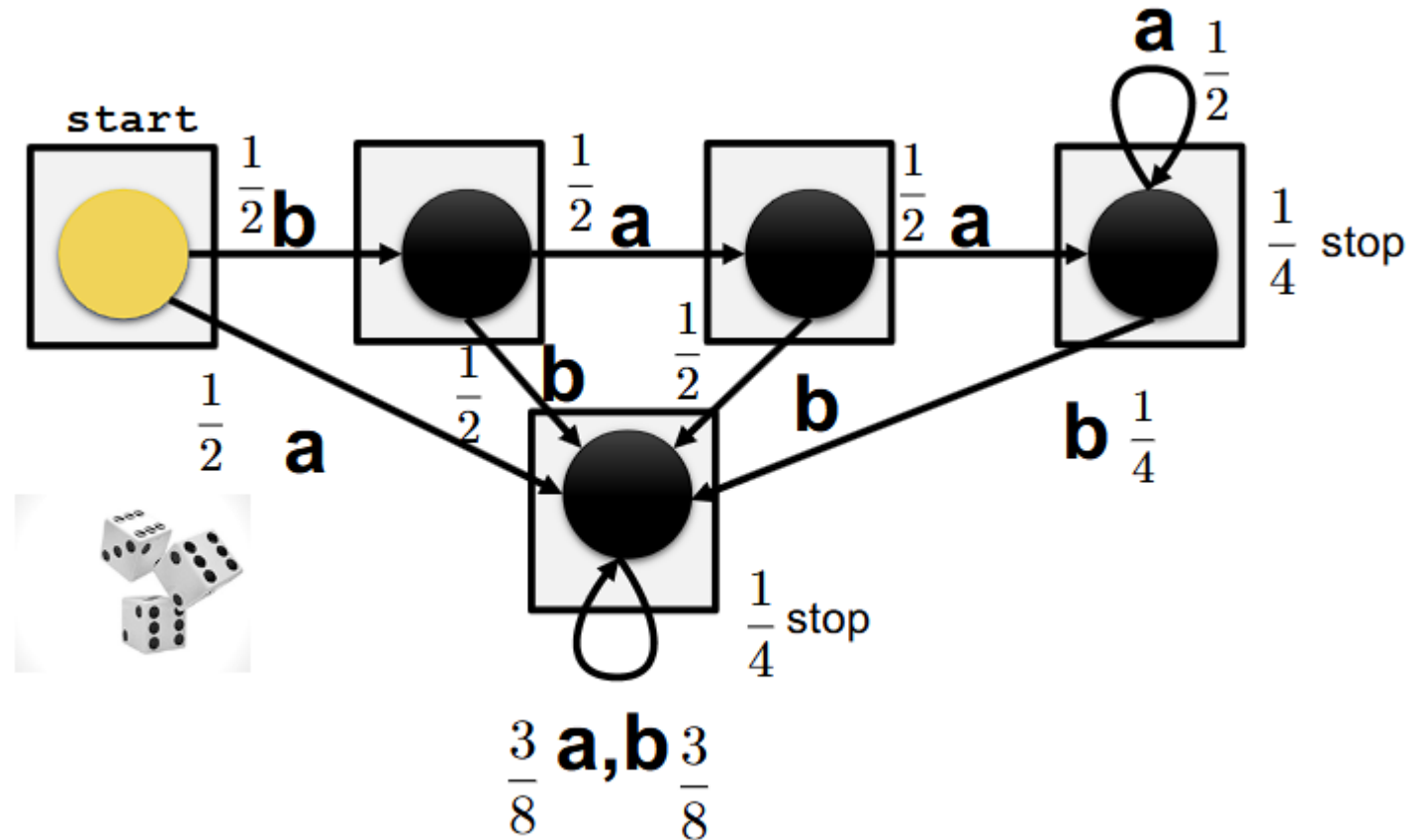
- Understand the Probability Ranking Principle
- Know and apply the RSV Formula
 - You do not need to know specific derivation steps!
- Know and apply Bayes Formula
 - Prior, Posterior, Evidence
- Know the other various formulas

Finite State Automata (FSA)



Generating Documents from FSA

Assign probabilities!



Generating Documents

“Probability that document D consists of $d_1d_2d_3d_4$ ”

$$\begin{aligned} P(D = (d_1, d_2, d_3, d_4)) = & \\ & (1 - p_{stop})P(D_1 = d_1) \\ & (1 - p_{stop})P(D_2 = d_2|D_1 = d_1) \\ & (1 - p_{stop})P(D_3 = d_3|D_1 = d_1 \wedge D_2 = d_2) \\ & (1 - p_{stop})P(D_4 = d_4|D_2 = d_2 \wedge D_3 = d_3)p_{stop} \end{aligned}$$

Unigram Model



t	p_t
to	0.3
be	0.2
or	0.1
not	0.05
that	0.15
is	0.1

Unigram model

Language Models – Exam Questions

- Calculate probabilities of generating sentences
- Beware of stop probabilities

Exam

- **5th of August, 14:00 – 17:00, Room TBD**
 - Be there 30 minutes before the exam starts
- 180 minutes is plenty of time! Double-check your answers.
- Everything talked about in the lectures, exercise sessions and in the assignments is relevant
 - Use the checklist on my website as a reference (outdated, but generally most concepts stayed the same)
- Fourny is a very nice lecturer. He doesn't want you to fail.
- Have fun studying and see you then!

Final Questions